# FirmApp - Extractos de código de los cambios pedidos

Desde la entrega de la pauta de la Evaluación N°3 hasta hoy. Cada extracto indica qué hace y dónde está; el código es el real del repositorio (generado desde los archivos, no reescrito). Para los cambios más grandes se muestran las piezas clave, no el archivo completo.

| # | Pedido | Commit |
|---|---|---|
| 1 | Pauta E3: registro + login reales y persistencia en JSON | f3a5036 |
| 2 | Remote Config en tiempo real (polling + archivo local) | fbbe957, 66105c3 |
| 3 | Revisión del proyecto: correcciones 1 a 5 | 200341b |
| 4 | Pedir Vacaciones, Solicitar Permiso, Subir Licencia + notificaciones | 0e8de55 |
| 5 | El botón de volver no funcionaba | db699f8 |
| 6 | Panel de administración para crear documentos | ab631e1 |
| 7 | Listas reales en los menús desplegables | d565d56 |
| 8 | Los menús desplegables no funcionaban | 11d6123 |

## 1. Autenticación real (registro + login) y persistencia en JSON

*La pauta exigía los 4 servicios: autenticación con registro y login, persistencia en JSON, notificaciones y Remote Config. Antes el login solo validaba el formato; aquí pasa a autenticar de verdad y a guardar datos.*

### Hash de contraseñas y clave canónica de cuenta

Nunca se guarda la contraseña en texto plano: se guarda su SHA-256. La clave de la cuenta se normaliza (correo en minúsculas; RUT solo dígitos y K) para que el mismo RUT en distinto formato sea la misma cuenta (esta normalización se añadió en la revisión, sección 3).

*data_manager.py*

```
def _hash_password(password):
    """Hash simple (SHA-256) para no guardar contrasenas en texto plano
    en el JSON. No es para produccion (faltaria sal por usuario, por
    ejemplo) -- alcanza para esta maqueta, y es mejor que texto plano."""
    return hashlib.sha256(password.encode('utf-8')).hexdigest()

def _normalize_key(identifier):
    """Clave canonica de una cuenta: el correo en minusculas, o el RUT solo
    con digitos y K mayuscula -- asi '12.345.678-5' y '123456785' son la
    misma cuenta (no se puede registrar dos veces en distinto formato)."""
    text = identifier.strip()
    if '@' in text:
        return text.lower()
    return ''.join(c for c in text if c.isdigit() or c in 'kK').upper()
```

### Cuentas precargadas (trabajador y administrador)

Al crearse el DataManager se agregan, si faltan, la cuenta de prueba y la de administrador, también en un usuarios.json ya existente.

*data_manager.py - DataManager.__init__*

```
def __init__(self):
    try:
        os.makedirs(DATA_DIR, exist_ok=True)
    except OSError as exc:
        print('DataManager: no se pudo crear la carpeta data/ (%s)' % exc)
    # Cuentas precargadas, para poder iniciar sesion sin registrarse antes
    # (el "Plan B Tecnico" de la Sesion 24 recomienda tener datos de
    # respaldo listos para la demo): un trabajador de prueba y la cuenta
    # de ADMINISTRADOR que crea los documentos por firmar. Se agregan solo
    # si faltan, asi tambien aparecen en un usuarios.json ya existente.
    users = self.load_users()
    seeded = False
    for key, password, role in (
            ('demo@firmapp.cl', 'demo1234', 'trabajador'),
            (ADMIN_ID, 'admin1234', 'admin')):
        if key not in users:
            users[key] = {'password_hash': _hash_password(password), 'role': role}
            seeded = True
    if seeded:
        self._save_users(users)
```

## Registrar, autenticar y consultar el rol

register_user rechaza duplicados y guarda la cuenta; authenticate compara el hash; get_role devuelve "admin" o "trabajador". Cada E/S va con try/except en load/_save.

*data_manager.py - DataManager*

```
def register_user(self, identifier, password):
    """Crea una cuenta nueva. Devuelve (ok, mensaje) -- ok=False si ya
    existe una cuenta con ese identificador o si fallo el guardado."""
    key = _normalize_key(identifier)
    users = self.load_users()
    if key in users:
        return False, 'Ya existe una cuenta con ese RUT o correo.'
    users[key] = {'password_hash': _hash_password(password), 'role': 'trabajador'}
    if not self._save_users(users):
        return False, 'No se pudo guardar la cuenta (error de disco).'
    return True, 'Cuenta creada correctamente.'

def authenticate(self, identifier, password):
    """True si el identificador y la contrasena coinciden con una
    cuenta registrada en usuarios.json."""
    key = _normalize_key(identifier)
    user = self.load_users().get(key)
    if not user:
        return False
    return user.get('password_hash') == _hash_password(password)

def get_role(self, identifier):
    """'admin' o 'trabajador' (las cuentas registradas desde la app son
    siempre 'trabajador': solo se puede ser admin editando usuarios.json)."""
    user = self.load_users().get(_normalize_key(identifier), {})
    return user.get('role', 'trabajador')
```

## Persistir y recuperar los documentos firmados

Lee data/documentos.json al abrir (si no existe o está corrupto usa los datos de ejemplo) y lo escribe cada vez que cambia el estado de los documentos.

*data_manager.py - DataManager*

```
def load_documents(self, default_pending, default_recent):
    """Devuelve (pending, recent) guardados en documentos.json. Si el
    archivo no existe todavia (primera vez que se abre la app), usa
    los datos de ejemplo de main.py como semilla inicial -- nunca
    devuelve los mismos objetos que recibio, para que mutarlos despues
    no afecte a esos valores por defecto."""
    try:
        with open(DOCS_FILE, 'r', encoding='utf-8') as f:
            data = json.load(f)
        return data.get('pending', default_pending), data.get('recent', default_recent)
    except FileNotFoundError:
        return list(default_pending), list(default_recent)
    except (IOError, OSError, json.JSONDecodeError) as exc:
        print('DataManager: no se pudo leer documentos.json (%s) -- '
              'se usan los datos de ejemplo' % exc)
        return list(default_pending), list(default_recent)

def save_documents(self, pending, recent):
    """Guarda el estado actual de PENDING/RECENT. Se llama cada vez
    que se firma un documento (InterfazRRHHApp.complete_signature())."""
    try:
        with open(DOCS_FILE, 'w', encoding='utf-8') as f:
            json.dump({'pending': pending, 'recent': recent}, f,
                      indent=2, ensure_ascii=False)
        return True
    except (IOError, OSError) as exc:
        print('DataManager: no se pudo guardar documentos.json (%s)' % exc)
        return False
```

## Carga inicial de PENDING/RECENT desde el JSON

Al importar main.py los documentos se cargan del JSON: un documento firmado sigue firmado tras cerrar y reabrir.

*main.py (nivel de módulo)*

```
DM = DataManager()
PENDING, RECENT = DM.load_documents(DEFAULT_PENDING, DEFAULT_RECENT)
```

### Login: autenticar contra las cuentas guardadas

Si el formato es válido pero la cuenta o la clave no coinciden, se muestra un error y no se navega.

*main.py - LoginScreen*

```
def login(self):
    app = MDApp.get_running_app()
    identifier = self.ids.identifier.text
    password = self.ids.password.text
    if app.dm.authenticate(identifier, password):
        self.ids.auth_feedback.text = ''
        app.go_to_home_after_login(identifier)
    else:
        self.ids.auth_feedback.text = (
            'RUT/correo o contraseña incorrectos. '
            '¿Todavía no tienes cuenta? Regístrate abajo.')
```

### Registro de cuenta nueva

Crea la cuenta con DataManager.register_user() y, si sale bien, entra directo a Inicio (registro + login en un paso).

*main.py - RegisterScreen*

```
def register(self):
    app = MDApp.get_running_app()
    identifier = self.ids.identifier.text
    password = self.ids.password.text
    ok, mensaje = app.dm.register_user(identifier, password)
    self.ids.feedback.text = mensaje
    self.ids.feedback.text_color = (T.C_PRIMARY if ok else
                                    T.rgba(T.ORANGE, .95))
    if ok:
        app.go_to_home_after_login(identifier)
```

### Firmar deja un registro real

El documento pasa de PENDING a RECENT y se guarda en JSON; luego se refresca el tablero.

*main.py - InterfazRRHHApp*

```
def complete_signature(self, doc_type, title, deadline, pages):
    """Pasa un documento de PENDING a RECENT y lo persiste en JSON
    (data_manager.py) -- Criterio B de la Sesion 24: el documento
    firmado sigue firmado aunque se cierre y se vuelva a abrir la app.
    Llamado por SignPanel.sign() antes de navegar a Exito."""
    PENDING[:] = [d for d in PENDING if d['title'] != title]
    RECENT.insert(0, dict(
        name=title,
        date=datetime.now().strftime('%d %b %Y'),
        filesize='—',
        offline=True,
    ))
    self.dm.save_documents(PENDING, RECENT)
    dashboard = self.home_screen.ids.dashboard
    dashboard.refresh_pending()
    dashboard.set_max_recent(dashboard.max_recent)
```

## 2. Remote Config en tiempo real

*Pedido: que un cambio en el archivo se refleje mientras la app corre, sin reiniciar y sin hacer push.*

### Interruptor local/GitHub y periodo de consulta

Con REMOTE_CONFIG_USE_LOCAL = True la app lee remote/config.json del disco (cambio visible en <= 5 s, sin push ni CDN). Con False lee la URL de GitHub.

*main.py (constantes)*

```
REMOTE_CONFIG_URL = ('https://raw.githubusercontent.com/zikness/FirmApp/'
                     'main/remote/config.json')
REMOTE_CONFIG_POLL_INTERVAL = 5  # segundos entre cada re-consulta

# Simulacion de la respuesta de RR.HH.: como no hay backend, cada solicitud se
# 'aprueba' sola despues de estos segundos y genera la notificacion.
REVIEW_DELAY = 10

# Interruptor de la demo en vivo: True = lee remote/config.json del disco
# (editas, guardas, y en <=5 s cambia la app, sin push ni CDN); False =
# lee la URL de GitHub de arriba (fuente remota real, requiere push).
REMOTE_CONFIG_USE_LOCAL = True
REMOTE_CONFIG_LOCAL_PATH = os.path.join(os.path.dirname(os.path.abspath(__file__)),
                                        'remote', 'config.json')
if REMOTE_CONFIG_USE_LOCAL:
    REMOTE_CONFIG_URL = REMOTE_CONFIG_LOCAL_PATH
```

## Polling: volver a pedir el valor cada pocos segundos

Se agenda con Clock.schedule_interval al construir la app; cada ciclo repite el fetch.

*main.py - InterfazRRHHApp*

```
remote_config.fetch(REMOTE_CONFIG_URL, on_loaded=self._apply_remote_config)
# Vuelve a pedir el mismo valor cada pocos segundos (ver comentario
# junto a REMOTE_CONFIG_POLL_INTERVAL): asi, si el archivo remoto
# cambia mientras la app sigue abierta, el banner/CTA/limite se
# actualizan solos, sin reiniciar la app.
Clock.schedule_interval(self._poll_remote_config, REMOTE_CONFIG_POLL_INTERVAL)

self.refresh_balance()

# El aviso emergente ('Sus vacaciones fueron aprobadas') va arriba de
# TODAS las pantallas, por eso la raiz es un FloatLayout que contiene
# al ScreenManager y al aviso.
root = FloatLayout()
root.add_widget(self.sm)
self.toast = Factory.NotificationToast()
root.add_widget(self.toast)

def _poll_remote_config(self, dt):
    remote_config.fetch(REMOTE_CONFIG_URL, on_loaded=self._apply_remote_config)
```

## fetch(): fuente HTTP o archivo local

Si la fuente es http(s) se usa UrlRequest (asíncrono); si no, se lee el archivo y el resultado se entrega en el siguiente tick del Clock, para que quien llama no distinga un caso del otro. Falla -> Fallback.

*remote_config.py - fetch()*

```
ruta_local = url[len('file://'):] if url.startswith('file://') else url
es_url_http = url.startswith('http://') or url.startswith('https://')

if es_url_http:
    UrlRequest(url, on_success=_on_success, on_error=_on_error,
               on_failure=_on_failure, timeout=timeout)
    return

# Fuente local (archivo de configuracion en disco): se lee de forma
# sincrona -- es solo E/S local, no hay por que esperar un hilo de red
# -- pero el resultado se entrega via Clock.schedule_once para que
# on_loaded()/on_failed() siempre se llamen "en el siguiente tick",
# igual que si hubieran venido de UrlRequest (asi quien llama a
# fetch() no distingue un caso del otro).
try:
    with open(ruta_local, 'r', encoding='utf-8') as f:
        contenido = f.read()
except (IOError, OSError) as exc:
    Logger.warning('RemoteConfig: no se pudo leer "%s" (%s) -- se '
                   'mantiene el valor local (Fallback)', ruta_local, exc)
    if on_failed:
        Clock.schedule_once(lambda dt: on_failed(), 0)
    return
Clock.schedule_once(lambda dt: _aplicar(contenido), 0)
```

## Validación de valores (aplicada en la revisión)

Cada valor se valida/convierte; si es inválido ("3" entre comillas, null) se ignora y se mantiene el anterior.

*main.py*

```
def _coerce_bool(value):
    """True/False si 'value' es un booleano o 'true'/'false' (texto); None si no."""
    if isinstance(value, bool):
        return value
    if isinstance(value, str) and value.strip().lower() in ('true', 'false'):
        return value.strip().lower() == 'true'
    return None

def _coerce_int(value):
    """Entero >= 0 si 'value' es un numero o un texto numerico; None si no."""
    if isinstance(value, bool):
        return None
    if isinstance(value, (int, float)) or (isinstance(value, str) and value.strip().lstrip('-').isdigit()):
        try:
            return max(0, int(value))
        except (ValueError, OverflowError):
            return None
    return None
```

## Aplicar solo lo que cambió

Evita reconstruir listas cada 5 s: solo se toca la pantalla si el valor realmente cambió. También sincroniza el aviso de notificaciones si está abierto.

*main.py - InterfazRRHHApp*

```
def _apply_remote_config(self, cfg):
    """Se llama SOLO si remote_config.fetch() trajo un JSON valido
    (Activate). Si nunca se llama -- sin internet, servidor caido, JSON
    todavia no existe en GitHub -- la app sigue con lo que ya tenia
    (Fallback), sin que haga falta ningun 'else' aqui.

    Como el valor se re-consulta cada pocos segundos (polling), dos
    cuidados: (1) cada valor se valida/convierte y, si es invalido
    (ej. "3" entre comillas o null por un error de tipeo al editar el
    JSON en vivo), se ignora y se mantiene el anterior; (2) solo se
    toca la pantalla si el valor realmente cambio, para no reconstruir
    las listas cada 5 segundos."""
    dashboard = self.home_screen.ids.dashboard
    applied = self._remote_applied

    banner = cfg.get('banner_message')
    if not isinstance(banner, str):
        Logger.warning('RemoteConfig: banner_message invalido (%r) -- se ignora', banner)
    elif applied.get('banner_message') != banner:
        applied['banner_message'] = banner
        dashboard.banner_text = banner
        if self.notification_overlay:
            self.notification_overlay.message = banner

    cta = _coerce_bool(cfg.get('show_desktop_cta'))
    if cta is None:
        Logger.warning('RemoteConfig: show_desktop_cta invalido (%r) -- se ignora',
                       cfg.get('show_desktop_cta'))
    elif applied.get('show_desktop_cta') != cta:
        applied['show_desktop_cta'] = cta
        dashboard.set_desktop_cta_visible(cta)

    max_recent = _coerce_int(cfg.get('max_recent_docs'))
    if max_recent is None:
        Logger.warning('RemoteConfig: max_recent_docs invalido (%r) -- se ignora',
                       cfg.get('max_recent_docs'))
    elif applied.get('max_recent_docs') != max_recent:
        applied['max_recent_docs'] = max_recent
        dashboard.set_max_recent(max_recent)
```

# 3. Revisión del proyecto (correcciones aprobadas)

*Cambios 1 a 5 de la revisión; los de Remote Config y la normalización ya se muestran arriba.*

## Limpieza de la sesión

Cerrar sesión limpia el formulario del login (no debe quedar la clave) y vuelve a Login.

*main.py*

```
def reset(self):
    """Limpia el formulario (al cerrar sesion no debe quedar la clave)."""
    self.ids.identifier.text = ''
    self.ids.password.text = ''
    self.ids.auth_feedback.text = ''
    self._id_valid = self._password_ok = False
    self._refresh_cta()

def logout(self):
    """Cierra la sesion y vuelve al Login (limpiando el formulario)."""
    self.current_user = ''
    self.hide_notifications()
    self._hide_toast()
    self.login_screen.reset()
    self.sm.current = 'login'
```

# 4. Pedir Vacaciones, Solicitar Permiso y Subir Licencia

*Tres pantallas nuevas con calendario, motivo, menú desplegable y archivo/foto, más notificaciones que abren la sección.*

## Reglas de negocio (sin Kivy): vacaciones y permiso

Vacaciones: rango completo, desde hoy, dentro del saldo y con motivo. Permiso: de 1 a 3 días (no son vacaciones).

*solicitudes.py*

```
def validate_vacation(start, end, reason, available, today=None):
    """(ok, mensaje). Rango completo, desde hoy, dentro del saldo."""
    today = today or date.today()
    if not start or not end:
        return False, 'Elige el primer y el último día en el calendario.'
    if start < today:
        return False, 'No puedes pedir vacaciones en fechas pasadas.'
    if end < start:
        return False, 'La fecha de término es anterior a la de inicio.'
```

```
    if count_days(start, end) > available:
        return False, 'Solo tienes %s disponibles.' % plural_days(available)
    err = _check_reason(reason)
    return (False, err) if err else (True, '')

def validate_permit(start, end, reason, today=None):
    """(ok, mensaje). Un permiso es de 1 a PERMIT_MAX_DAYS dias."""
    today = today or date.today()
    if not start or not end:
        return False, 'Elige el día (o los días) en el calendario.'
    if start < today:
        return False, 'No puedes pedir un permiso en fechas pasadas.'
    if count_days(start, end) > PERMIT_MAX_DAYS:
        return False, 'Un permiso es de 1 a %d días.' % PERMIT_MAX_DAYS
    err = _check_reason(reason)
    return (False, err) if err else (True, '')
```

## Validación de la licencia

Comprueba que sea un documento utilizable: extensión permitida, contenido real igual a la extensión (magic bytes), peso 10 KB-10 MB, PDF completo (%%EOF) y resolución mínima en imágenes. No verifica autenticidad (requeriría OCR).

*solicitudes.py*

```
def _magic_matches(head, ext):
    if ext == '.pdf':
        return head.startswith(b'%PDF')
    if ext == '.png':
        return head.startswith(b'\x89PNG\r\n\x1a\n')
    return head.startswith(b'\xff\xd8\xff')          # .jpg / .jpeg

def validate_license_file(path, from_camera=False, image_size=None):
    """
    Valida que el archivo sirva como licencia. (ok, mensaje).

    OJO -- alcance real: sin un backend ni OCR, la app NO puede leer el
    contenido ni comprobar que el documento sea autentico (eso lo revisa
    RR.HH. / COMPIN). Lo que SI verifica es que sea un documento utilizable:
      1. existe y es un archivo,
      2. tiene formato permitido (PDF, PNG o JPG; solo imagen si viene de la
         camara),
      3. su contenido real coincide con la extension (no es un .exe o un
         .txt renombrado a .pdf),
      4. su peso es razonable (10 KB a 10 MB),
      5. si es imagen, tiene resolucion suficiente para leerse
         (image_size = (ancho, alto), lo mide quien llama con Kivy),
      6. si es PDF, esta completo (trae la marca final %%EOF).
    """
    if not path or not os.path.isfile(path):
        return False, 'No se encontró el archivo.'
    ext = os.path.splitext(path)[1].lower()
    allowed = IMAGE_EXTS if from_camera else LICENSE_EXTS
    if ext not in allowed:
        return False, ('Formato no permitido. Sube un PDF, PNG o JPG.'
                       if not from_camera else 'La foto debe ser PNG o JPG.')
    size = os.path.getsize(path)
    if size < LICENSE_MIN_BYTES:
        return False, 'El archivo es muy liviano para ser una licencia.'
    if size > LICENSE_MAX_BYTES:
        return False, 'El archivo supera los 10 MB.'
    try:
        with open(path, 'rb') as f:
            head = f.read(16)
            if ext == '.pdf':
                f.seek(max(0, size - 2048))
                tail = f.read()
    except OSError:
        return False, 'No se pudo leer el archivo.'
    if not _magic_matches(head, ext):
        return False, 'El contenido no corresponde a un %s válido.' % ext[1:].upper()
    if ext == '.pdf' and b'%%EOF' not in tail:
        return False, 'El PDF está incompleto o dañado.'
    if ext != '.pdf':
        if not image_size:
            return False, 'No se pudo abrir la imagen.'
        if min(image_size) < LICENSE_MIN_SIDE:
            return False, ('La imagen es muy pequeña para leerse. '
                           'Toma la foto más de cerca.')
    return True, 'Documento válido: formato, peso y legibilidad correctos.'
```

## Registro de solicitud y texto de la notificación

Crea el registro (estado pendiente) y arma el mensaje "Sus vacaciones fueron aprobadas (fechas)".

*solicitudes.py*

```
KINDS = {
    'vacaciones': dict(label='Vacaciones', icon='vacation',
                       approved='Sus vacaciones fueron aprobadas'),
    'permiso': dict(label='Permiso', icon='permit',
                    approved='Su permiso fue aprobado'),
    'licencia': dict(label='Licencia', icon='camera',
                     approved='Su licencia fue validada y aprobada'),
}

def new_request(kind, **fields):
    """Registro de una solicitud, listo para guardarse en JSON."""
    return dict(id=uuid.uuid4().hex[:8], kind=kind, status='pendiente',
                created=datetime.now().isoformat(timespec='seconds'), **fields)

def approval_message(req):
    """Texto de la notificacion cuando RR.HH. responde."""
    base = KINDS[req['kind']]['approved']
    if req['kind'] == 'licencia':
        return base
    start, end = to_date(req['start']), to_date(req['end'])
    return '%s (%s)' % (base, fmt_range(start, end))
```

## Calendario de rango (CalendarPicker)

Componente propio: 1er toque = primer día, 2do = último (el mismo día = un solo día). Deshabilita días pasados y los que superarían max_days (3 en permiso; el saldo en vacaciones).

*widgets.py*

```
def pick(self, d):
    if d < self._today:
        return
    if self.single:
        self.start = self.end = d
    elif self.start is None or self.end is not None:
        self.start, self.end = d, None
    elif d < self.start:
        self.start = d
    else:
        if self.max_days and count_days(self.start, d) > self.max_days:
            return
        self.end = d
    self.refresh()
    self.dispatch('on_range_change')

def _state_for(self, d):
    if d < self._today:
        return 'disabled'
    if d == self.start or d == self.end:
        return 'edge'
    if self.start and self.end and self.start < d < self.end:
        return 'range'
    if (self.start and not self.end and self.max_days and d > self.start
            and count_days(self.start, d) > self.max_days):
        return 'disabled'
    return 'today' if d == self._today else 'normal'

def refresh(self):
    self.month_label = '%s %d' % (MESES_LARGO[self.month - 1], self.year)
    self.can_prev = (self.year, self.month) > (self._today.year, self._today.month)
    offset = date(self.year, self.month, 1).weekday()      # lunes = 0
    ndays = calendar.monthrange(self.year, self.month)[1]
    for i, cell in enumerate(self._cells):
        n = i - offset + 1
        if 1 <= n <= ndays:
            d = date(self.year, self.month, n)
            cell.day, cell.text, cell.state = d, str(n), self._state_for(d)
        else:
            cell.day, cell.text, cell.state = None, '', 'empty'
    self.complete = bool(self.start and self.end)
    if self.complete and self.single:
        self.summary = 'Fecha límite: %s' % fmt_day(self.start)
    elif self.complete:
        self.summary = '%s · %s' % (fmt_range(self.start, self.end),
                                    plural_days(count_days(self.start, self.end)))
    elif self.start:
        self.summary = 'Ahora toca el último día (o el mismo si es uno solo)'
    elif self.single:
        self.summary = 'Toca el día límite'
    else:
        self.summary = 'Toca el primer día'
```

## Una celda del calendario (kv)

El color de cada día depende de su estado: borde (hoy), relleno (extremos), tinte (dentro del rango), apagado (deshabilitado).

*interfazrrhh.kv -*

```
<DayCell>:
    radius: dp(12)
    md_bg_color: T.C_PRIMARY if root.state == 'edge' else (T.C_TINT_20 if root.state == 'range' else (0, 0, 0, 0))
    line_color: T.C_PRIMARY if root.state == 'today' else (0, 0, 0, 0)
    line_width: 1.5
    disabled: root.state in ('empty', 'disabled')
    TxtFijo:
        text: root.text
        font_name: T.FONT
        font_size: sp(13)
        bold: root.state in ('edge', 'today')
        text_color: (1, 1, 1, 1) if root.state == 'edge' else (T.C_PRIMARY_DARK if root.state == 'range' else
            (T.rgba(T.SECONDARY, .55) if root.state == 'disabled' else T.C_TEXT))
        disabled_color: self.text_color
```

## Menú desplegable en línea (DropdownField)

Un campo que al tocarlo abre la lista de opciones debajo (con descripción por opción) y avisa on_select. Sin ventana flotante, para respetar la paleta y funcionar dentro de un ScrollView.

*widgets.py*

```
class DropdownField(MDBoxLayout):
    """
    Menu desplegable en linea: un campo que al tocarlo abre la lista de
    opciones justo debajo (sin ventana flotante, para que respete la paleta
    y funcione igual dentro de un ScrollView). 'selected' guarda la opcion
    elegida y on_select avisa cuando cambia.
    """
    __events__ = ('on_select',)

    options = ListProperty([])
    details = DictProperty({})          # descripcion opcional de cada opcion
    placeholder = StringProperty('Selecciona una opción')
    selected = StringProperty('')
    expanded = BooleanProperty(False)

    def on_kv_post(self, base_widget):
        self._build_options()

    def on_options(self, *args):
        if 'options_box' in self.ids:
            self._build_options()

    def on_details(self, *args):
        if 'options_box' in self.ids:
            self._build_options()

    def _build_options(self):
        box = self.ids.options_box
        box.clear_widgets()
        for opt in self.options:
            box.add_widget(OptionRow(text=opt, owner=self,
                                     detail=self.details.get(opt, ''),
                                     selected=(opt == self.selected)))

    def toggle(self):
        self.expanded = not self.expanded

    def choose(self, text):
        self.selected = text
        self.expanded = False
        for row in self.ids.options_box.children:
            row.selected = (row.text == text)
        self.dispatch('on_select', text)

    def reset(self):
        self.selected = ''
        self.expanded = False
        for row in self.ids.options_box.children:
            row.selected = False

    def on_select(self, *args):
        pass
```

## Pantalla de Vacaciones

Limita el calendario al saldo, valida y envía. PermitScreen es análoga (max_days = 3, chips de motivos).

*main.py*

```
class VacationScreen(RequestScreen):
```

```
    """
    'Pedir Vacaciones': calendario para elegir el rango + motivo. El saldo de
    dias disponibles limita el calendario (CalendarPicker.max_days) y
    solicitudes.validate_vacation() lo vuelve a comprobar al enviar.
    """
    kind = 'vacaciones'
    available = NumericProperty(0)
    error = StringProperty('')

    def on_kv_post(self, base_widget):
        self.ids.cta.set_enabled(False)

    def reset(self):
        self.available = MDApp.get_running_app().vacation_available()
        cal = self.ids.calendar
        cal.max_days = max(1, self.available)
        cal.clear()
        self.ids.reason.clear()
        self.refresh()

    def refresh(self):
        cal = self.ids.calendar
        ok, msg = solicitudes.validate_vacation(
            cal.start, cal.end, self.ids.reason.text, self.available)
        self.error = msg if (cal.complete and msg) else ''
        self.ids.cta.set_enabled(ok)

    def send(self):
        cal = self.ids.calendar
        reason = self.ids.reason.text.strip()
        ok, msg = solicitudes.validate_vacation(cal.start, cal.end, reason, self.available)
        if not ok:
            self.error = msg
            return
        MDApp.get_running_app().submit_request(
            'vacaciones', start=cal.start.isoformat(), end=cal.end.isoformat(),
            days=solicitudes.count_days(cal.start, cal.end), reason=reason)
```

## Pantalla de Licencia: subir archivo o tomar foto

Usa plyer.filechooser y plyer.camera; sin cámara avisa y deja elegir una foto; sin selector del sistema abre un selector propio. Valida al elegir.

*main.py - LicenseScreen*

```
def pick_file(self):
    self.source = 'file'
    self.notice = ''
    self._open_picker(solicitudes.LICENSE_EXTS)

def take_photo(self):
    self.source = 'camera'
    self.notice = ''
    path = os.path.join(tempfile.gettempdir(), 'firmapp_licencia.jpg')
    try:
        from plyer import camera
        camera.take_picture(
            filename=path,
            on_complete=lambda p: Clock.schedule_once(
                lambda dt: self._on_selected([p]), 0))
    except Exception as exc:
        Logger.info('Licencia: sin camara disponible (%s)', exc)
        self.notice = ('No se detectó una cámara en este equipo. '
                       'Elige una foto guardada.')
        self._open_picker(solicitudes.IMAGE_EXTS)

def _open_picker(self, exts):
    try:
        from plyer import filechooser
        filechooser.open_file(
            title='Selecciona tu licencia',
            filters=['*' + e for e in exts],
            on_selection=lambda sel: Clock.schedule_once(
                lambda dt: self._on_selected(sel), 0))
    except Exception as exc:
        Logger.info('Licencia: selector del sistema no disponible (%s)', exc)
        FilePickerPopup(exts, self._on_selected).open()

def _on_selected(self, selection):
    if not selection:
        return
    path = selection[0]
    ext = os.path.splitext(path)[1].lower()
    image_size = None
    if ext in solicitudes.IMAGE_EXTS:
        try:
            from kivy.core.image import Image as CoreImage
```

```
            image_size = CoreImage(path).size
        except Exception:
            image_size = None
    ok, msg = solicitudes.validate_license_file(
        path, from_camera=(self.source == 'camera'), image_size=image_size)
    self.file_path = path
    self.file_name = os.path.basename(path)
    self.file_is_image = ext in solicitudes.IMAGE_EXTS
    self.file_ok = ok
    self.file_msg = msg
    self.refresh()
```

## Enviar una solicitud y agendar la respuesta

Guarda la solicitud en JSON, avisa que quedó en revisión y agenda la respuesta simulada de RR.HH. (no hay backend). La licencia se copia a data/licencias/.

*main.py - InterfazRRHHApp*

```
def submit_request(self, kind, **fields):
    """Registra la solicitud (JSON), avisa que quedo en revision y agenda
    la respuesta simulada de RR.HH. (_approve)."""
    if kind == 'licencia':
        fields['stored_file'] = self._store_license_file(fields.pop('file_path'))
    req = solicitudes.new_request(kind, **fields)
    self.requests.append(req)
    self._persist_requests()
    self.refresh_balance()
    self._schedule_reviews()
    label = solicitudes.KINDS[kind]['label']
    resumen = solicitudes.request_title(req)
    notifications.notify('Solicitud enviada', '%s: %s' % (label, resumen))
    self.go_to_request_success(
        '¡Solicitud enviada!',
        'Tu solicitud de %s (%s) quedó en revisión. Te avisaremos apenas '
        'RR.HH. responda.' % (label.lower(), resumen))

def _schedule_reviews(self):
    """Agenda la respuesta de RR.HH. de cada solicitud pendiente (una vez)."""
    for req in self.requests:
        if req['status'] == 'pendiente' and req['id'] not in self._review_scheduled:
            self._review_scheduled.add(req['id'])
            Clock.schedule_once(lambda dt, rid=req['id']: self._approve(rid), REVIEW_DELAY)
```

## La respuesta de RR.HH. genera la notificación

Pasa la solicitud a "aprobada", crea la notificación, la persiste y la muestra (aviso emergente, campana, sistema).

*main.py - InterfazRRHHApp*

```
def _approve(self, req_id):
    """Respuesta simulada de RR.HH.: aprueba la solicitud y genera la
    notificacion (aviso emergente + campana + notificacion del sistema)."""
    req = next((r for r in self.requests if r['id'] == req_id), None)
    if not req or req['status'] != 'pendiente':
        return
    req['status'] = 'aprobada'
    notif = dict(id=req_id, kind=req['kind'], message=solicitudes.approval_message(req))
    self.notifications.append(notif)
    self.notif_count = len(self.notifications)
    self._persist_requests()
    notifications.notify('RR.HH. respondió tu solicitud', notif['message'])
    if self.current_user and self.sm.current != 'admin':
        self.show_toast(notif)
    self.refresh_overlay()
    current = self.sm.get_screen(self.sm.current)
    if isinstance(current, RequestScreen):
        current.refresh_requests()

def show_toast(self, notif):
    toast = self.toast
    toast.notif_id = notif['id']
    toast.text = notif['message']
    toast.icon_name = solicitudes.KINDS[notif['kind']]['icon']
    Animation.cancel_all(toast)
    toast.shown = True
    Animation(opacity=1, d=0.25, t='out_quad').start(toast)
    Clock.unschedule(self._hide_toast)
    Clock.schedule_once(self._hide_toast, 7)
```

## Tocar la notificación abre la sección y la elimina

Quita la notificación de la lista, cierra el aviso y navega a Vacaciones, Permiso o Licencia según su tipo.

*main.py - InterfazRRHHApp*

```
def open_notification(self, notif_id):
    """Al tocar una notificacion: se quita de la lista y se abre la
    seccion correspondiente (Vacaciones, Permiso o Licencia)."""
    notif = next((n for n in self.notifications if n['id'] == notif_id), None)
```

```
    if not notif:
        return
    self.notifications.remove(notif)
    self.notif_count = len(self.notifications)
    self._persist_requests()
    if self.toast.notif_id == notif_id:
        self._hide_toast()
    if self.notification_overlay:
        self.hide_notifications()
    {'vacaciones': self.go_to_vacations, 'permiso': self.go_to_permit,
     'licencia': self.go_to_license}[notif['kind']]()

def refresh_overlay(self):
    """Repuebla la lista de notificaciones del aviso grande (si esta abierto)."""
    overlay = self.notification_overlay
    if not overlay:
        return
    box = overlay.ids.notif_list
    box.clear_widgets()
    for n in self.notifications:
        box.add_widget(NotificationRow(
            notif_id=n['id'], text=n['message'],
            icon_name=solicitudes.KINDS[n['kind']]['icon']))
    overlay.item_count = len(self.notifications)
```

## Aviso emergente de notificación (kv)

Tarjeta fija arriba de todas las pantallas; al tocarla llama a open_notification.

*interfazrrhh.kv -*

```
<NotificationToast>:
    orientation: 'horizontal'
    size_hint: None, None
    size: dp(392), dp(66)
    pos_hint: {'center_x': .5, 'top': .985}
    opacity: 0
    radius: dp(20)
    padding: dp(12), 0
    spacing: dp(12)
    md_bg_color: 1, 1, 1, .98
    line_color: T.rgba(T.ORANGE, .55)
    line_width: 1.5
    shadow_strength: 2.6
    on_release: app.open_notification(root.notif_id)
    MDFloatLayout:
        size_hint: None, None
        size: dp(40), dp(40)
        pos_hint: {'center_y': .5}
        GradientTile:
            pos_hint: {'x': 0, 'y': 0}
            size_hint: 1, 1
            tile_radius: dp(13)
            c1: T.rgba(T.ORANGE)
            c2: T.rgba(T.ORANGE_DOT)
        Icon:
            name: root.icon_name
            color: 1, 1, 1, 1
            size_hint: None, None
            size: dp(20), dp(20)
            pos_hint: {'center_x': .5, 'center_y': .5}
    MDBoxLayout:
        orientation: 'vertical'
        spacing: dp(2)
        MDWidget:
        Txt:
            text: 'RR.HH. RESPONDIÓ · TOCA PARA ABRIR'
            font_size: sp(8.5)
            bold: True
            text_color: T.rgba(T.ORANGE)
        Txt:
            text: root.text
            font_size: sp(12)
            bold: True
            text_color: T.C_PRIMARY_DARK
        MDWidget:
    Icon:
        name: 'chevron'
        color: T.rgba(T.ORANGE)
        size_hint: None, None
        size: dp(14), dp(14)
        pos_hint: {'center_y': .5}
```

### Conexión de las tarjetas de Acciones Rápidas y el punto de la campana

Cada tarjeta abre su pantalla; el punto naranja de la campana depende de si hay notificaciones o banner.

*interfazrrhh.kv*

```
    label_text: 'Pedir Vacaciones'
    sub_text: 'Solicita días libres'
    c1: T.C_PRIMARY
    c2: T.C_PRIMARY_DARK
    on_release: app.go_to_vacations()
QuickActionCard:
    icon_name: 'permit'
    label_text: 'Solicitar Permiso'
    sub_text: 'Permiso administrativo'
    c1: T.C_PRIMARY_LIGHT
    c2: T.C_PRIMARY
    on_release: app.go_to_permit()
QuickActionCard:
    icon_name: 'camera'
    ...

                has_alerts: bool(root.banner_text) or app.notif_count > 0
```

## 5. El botón de volver no funcionaba

*Causa: el aviso emergente, invisible pero deshabilitado, absorbía los toques de la franja superior.*

### El aviso solo recibe toques mientras está visible

Se reemplazó disabled=True por una propiedad shown y se sobreescriben los eventos táctiles.

*widgets.py - NotificationToast*

```
class NotificationToast(PressCard):
    """Aviso emergente arriba de la pantalla cuando RR.HH. responde."""
    notif_id = StringProperty('')
    icon_name = StringProperty('check')
    text = StringProperty('')
    shown = BooleanProperty(False)

    # Vive siempre encima de todas las pantallas; oculto no debe absorber
    # toques (antes, con disabled=True, tapaba el boton de volver).
    def on_touch_down(self, touch):
        return self.shown and super().on_touch_down(touch)

    def on_touch_move(self, touch):
        return self.shown and super().on_touch_move(touch)

    def on_touch_up(self, touch):
        return self.shown and super().on_touch_up(touch)
```

## 6. Panel de administración para crear documentos

*Cuenta admin@firmapp.cl / admin1234: al iniciar sesión entra al panel en vez del Inicio.*

### Redirección según el rol

Si la cuenta es admin entra al panel; si no, al Inicio de trabajador.

*main.py - InterfazRRHHApp*

```
def go_to_home_after_login(self, identifier=''):
    """Login o Registro -> Inicio (o al panel, si la cuenta es admin).
    Llamado por LoginScreen.login() y por RegisterScreen.register() cuando
    la cuenta se crea con exito (registro + login combinados)."""
    self.current_user = identifier
    if self.dm.get_role(identifier) == 'admin':
        self.admin_screen.reset()
        self.sm.current = 'admin'
        return
    self.sm.current = 'home'
    self._schedule_reviews()
    # Notificaciones (Plyer): si al entrar ya hay un documento urgente,
    # se avisa una vez al sistema operativo -- el mismo lugar donde
    # iria una notificacion push real si el proyecto tuviera backend.
    # Va aqui (despues de "iniciar sesion") y no en build(), porque no
    # tendria sentido notificar antes de que el usuario se identifique.
    if any(doc.get('urgent') for doc in PENDING):
        notifications.notify(
            'Tienes documentos urgentes',
            'Hay %d documento(s) pendiente(s) por firmar.' %
            sum(1 for d in PENDING if d.get('urgent')))
```

## Validación de un documento nuevo

Tipo elegido, título único (5-80), fecha desde hoy y 1-200 páginas. El título debe ser único porque identifica al documento al firmarlo.

*solicitudes.py*

```
def validate_document(doc_type, title, deadline, pages_text, existing_titles, today=None):
    """(ok, mensaje). Reglas del panel de administracion al crear un
    documento por firmar: tipo elegido, titulo unico de 5 a 80 caracteres,
    fecha limite desde hoy y entre 1 y 200 paginas. El titulo debe ser unico
    porque identifica al documento al firmarlo (complete_signature)."""
    today = today or date.today()
    title = (title or '').strip()
    if not doc_type:
        return False, 'Elige el tipo de documento.'
    if not (DOC_TITLE_MIN <= len(title) <= DOC_TITLE_MAX):
        return False, 'El título debe tener entre %d y %d caracteres.' % (DOC_TITLE_MIN, DOC_TITLE_MAX)
    if title.lower() in (t.lower() for t in existing_titles):
        return False, 'Ya existe un documento con ese título.'
    if not deadline:
        return False, 'Elige la fecha límite en el calendario.'
    if deadline < today:
        return False, 'La fecha límite no puede ser anterior a hoy.'
    if not (pages_text or '').strip().isdigit() or not (1 <= int(pages_text) <= DOC_MAX_PAGES):
        return False, 'Las páginas deben ser un número entre 1 y %d.' % DOC_MAX_PAGES
    return True, ''
```

## Pantalla del panel

Valida en vivo, muestra el error solo cuando el formulario está completo, publica y lista los documentos.

*main.py - AdminScreen*

```
class AdminScreen(Screen):
    """
    Panel de administracion: solo entra la cuenta con rol 'admin'
    (data_manager.get_role). Desde aqui se CREAN los documentos que los
    trabajadores ven en 'Tareas Pendientes' y firman, y se pueden retirar.
    Los documentos se guardan en JSON (data/documentos.json), igual que los
    firmados, y las reglas de validacion estan en solicitudes.validate_document().
    """
    urgent = BooleanProperty(False)
    error = StringProperty('')
    success = StringProperty('')
    has_docs = BooleanProperty(False)
    docs_count = NumericProperty(0)

    def on_kv_post(self, base_widget):
        self.ids.cta.set_enabled(False)

    def reset(self):
        self.ids.type_select.reset()
        self.ids.title_in.clear()
        self.ids.pages_in.clear()
        self.ids.calendar.clear()
        self.urgent = False
        self.error = ''
        self.refresh()
        self.refresh_docs()

    def toggle_urgent(self):
        self.urgent = not self.urgent

    def _values(self):
        return (self.ids.type_select.selected, self.ids.title_in.text,
                self.ids.calendar.start, self.ids.pages_in.text)

    def refresh(self):
        doc_type, title, deadline, pages = self._values()
        ok, msg = solicitudes.validate_document(
            doc_type, title, deadline, pages, [d['title'] for d in PENDING])
        filled = bool(doc_type and title.strip() and deadline and pages.strip())
        self.error = '' if (ok or not filled) else msg
        self.ids.cta.set_enabled(ok)

    def create(self):
        doc_type, title, deadline, pages = self._values()
        ok, msg = solicitudes.validate_document(
            doc_type, title, deadline, pages, [d['title'] for d in PENDING])
        if not ok:
            self.error = msg
            return
        MDApp.get_running_app().create_document(
            doc_type, title.strip(), solicitudes.fmt_deadline(deadline),
            int(pages), self.urgent)
        self.reset()
        self.success = 'Documento "%s" publicado: ya aparece a los trabajadores.' % title.strip()
```

```
    def refresh_docs(self):
        box = self.ids.docs_box
        box.clear_widgets()
        for i, d in enumerate(PENDING):
            box.add_widget(AdminDocRow(
                title=d['title'], doc_type=d['doc_type'], urgent=d.get('urgent', False),
                detail='%s · Vence: %s · %s pág.' % (d['doc_type'], d['deadline'], d['pages']),
                last=(i == len(PENDING) - 1)))
        self.docs_count = len(PENDING)
        self.has_docs = bool(PENDING)
```

### Crear y retirar documentos

Agrega el documento a PENDING, lo guarda en JSON y refresca el tablero de los trabajadores.

*main.py - InterfazRRHHApp*

```
def create_document(self, doc_type, title, deadline, pages, urgent):
    """Publica un documento nuevo por firmar (lo crea el admin) y lo
    guarda en JSON; los trabajadores lo ven en 'Tareas Pendientes'."""
    PENDING.append(dict(doc_type=doc_type, title=title, deadline=deadline,
                        urgent=urgent, pages=pages))
    self.dm.save_documents(PENDING, RECENT)
    self.home_screen.ids.dashboard.refresh_pending()

def delete_document(self, title):
    PENDING[:] = [d for d in PENDING if d['title'] != title]
    self.dm.save_documents(PENDING, RECENT)
    self.home_screen.ids.dashboard.refresh_pending()
    self.admin_screen.refresh_docs()
    self.admin_screen.success = ''
```

### Calendario de un solo día

Para la fecha límite se reutiliza el CalendarPicker con single=True: un toque fija el día.

*widgets.py - CalendarPicker.pick (rama single)*

```
if self.single:
    self.start = self.end = d
elif self.start is None or self.end is not None:
    self.start, self.end = d, None
elif d < self.start:
    self.start = d
else:
    if self.max_days and count_days(self.start, d) > self.max_days:
        return
    self.end = d
```

## 7. Listas reales en los menús desplegables

*Se usaron las listas entregadas: 11 tipos de documento laboral y 7 tipos de licencia médica, con su descripción.*

### Tipos de licencia

Nombre corto + descripción; el menú muestra la descripción bajo cada opción.

*solicitudes.py*

```
_LICENSES = [
    ('Tipo 1 · Enfermedad o accidente común',
     'Cubre ausencias por patologías de origen no laboral ni profesional.'),
    ('Tipo 2 · Prórroga de medicina preventiva',
     'Para trabajadores sanos o en riesgo que deben someterse a exámenes o '
     'tratamientos que eviten el avance de enfermedades crónicas.'),
    ...
]
LICENSE_TYPES = [name for name, _ in _LICENSES]
LICENSE_DETAILS = dict(_LICENSES)
```

### Fila de opción con descripción (kv)

La fila crece según el largo de la descripción.

*interfazrrhh.kv -*

```
<OptionRow>:
    size_hint_y: None
    height: max(dp(46), opt_col.height + dp(20))
    padding: dp(14), dp(10)
    spacing: dp(8)
    radius: dp(12)
    md_bg_color: T.C_TINT_10 if root.selected else (0, 0, 0, 0)
    MDBoxLayout:
        id: opt_col
        orientation: 'vertical'
        size_hint_y: None
        height: self.minimum_height
        spacing: dp(2)
```

```
        pos_hint: {'center_y': .5}
        Txt:
            text: root.text
            font_size: sp(12)
            bold: True
            text_color: T.C_PRIMARY_DARK if root.selected else T.C_TEXT
        Txt:
            text: root.detail
            font_size: sp(9.5)
            text_color: T.C_SECONDARY
            opacity: 1 if root.detail else 0
            height: self.texture_size[1] if root.detail else 0
    Icon:
        name: 'check'
        color: T.C_PRIMARY
        opacity: 1 if root.selected else 0
        size_hint: None, None
        size: dp(14), dp(14)
        pos_hint: {'center_y': .5}
```

## 8. Los menús desplegables no funcionaban

*Causa: un contenedor con alto 0 no oculta a sus hijos; las filas del menú cerrado quedaban debajo del campo y, al estar disabled, consumían el toque.*

### Contenedores colapsados ignoran los toques

Si el alto es <= 1 el contenedor no pasa el toque a sus hijos. Se aplica a GlassCard, PressableBox y a la lista del menú.

*widgets.py*

```
class SkipWhenCollapsed:
    """
    Un contenedor "oculto" con height = 0 NO esconde a sus hijos: siguen
    dibujados fuera de su caja (solo se ven transparentes) y, aunque esten
    disabled, un widget disabled que choca con el toque lo CONSUME -- asi las
    filas de un menu cerrado tapaban el boton que lo abre. Este mixin hace
    que un contenedor colapsado (alto <= 1) ignore los toques sin pasarselos
    a los hijos.
    """
    def on_touch_down(self, touch):
        return False if self.height <= 1 else super().on_touch_down(touch)

    def on_touch_move(self, touch):
        return False if self.height <= 1 else super().on_touch_move(touch)

    def on_touch_up(self, touch):
        return False if self.height <= 1 else super().on_touch_up(touch)

class CollapsibleBox(SkipWhenCollapsed, MDBoxLayout):
    """MDBoxLayout que puede colapsar a alto 0 (lista del menu desplegable)."""
    pass
```

### Lista del menú (kv)

La lista pasó de MDBoxLayout a CollapsibleBox.

*interfazrrhh.kv -*

```
<DropdownField>:
    orientation: 'vertical'
    size_hint_y: None
    height: self.minimum_height
    spacing: dp(6)
    PressableBox:
        size_hint_y: None
        height: dp(56)
        padding: dp(16), 0
        spacing: dp(8)
        radius: dp(14)
        md_bg_color: 1, 1, 1, .7
        line_color: T.C_PRIMARY if (root.expanded or root.selected) else T.C_BG
        line_width: 1.8
        on_release: root.toggle()
        TxtFijo:
            text: root.selected if root.selected else root.placeholder
            shorten: True
            shorten_from: 'right'
            font_name: T.FONT
            font_size: sp(13)
            bold: bool(root.selected)
            text_color: T.C_TEXT if root.selected else T.rgba(T.SECONDARY, .85)
            halign: 'left'
            text_size: self.size
        Icon:
            name: 'down'
```

```
        color: T.C_PRIMARY
        size_hint: None, None
        size: dp(18), dp(18)
        pos_hint: {'center_y': .5}
CollapsibleBox:
    id: options_box
    orientation: 'vertical'
    size_hint_y: None
    height: (self.minimum_height) if root.expanded else 0
    opacity: 1 if root.expanded else 0
    disabled: not root.expanded
    padding: dp(6)
    spacing: dp(2)
    radius: dp(14)
    md_bg_color: 1, 1, 1, .85
    line_color: T.C_BG
    line_width: 1.2
```